



ELISA

Enabling **Linux** in
Safety Applications

A PoC Based Framework to Evaluate Linux VMA Criticalities in Safety-Critical Systems

Alessandro Carminati - NVIDIA

ELISA Workshop: London, UK

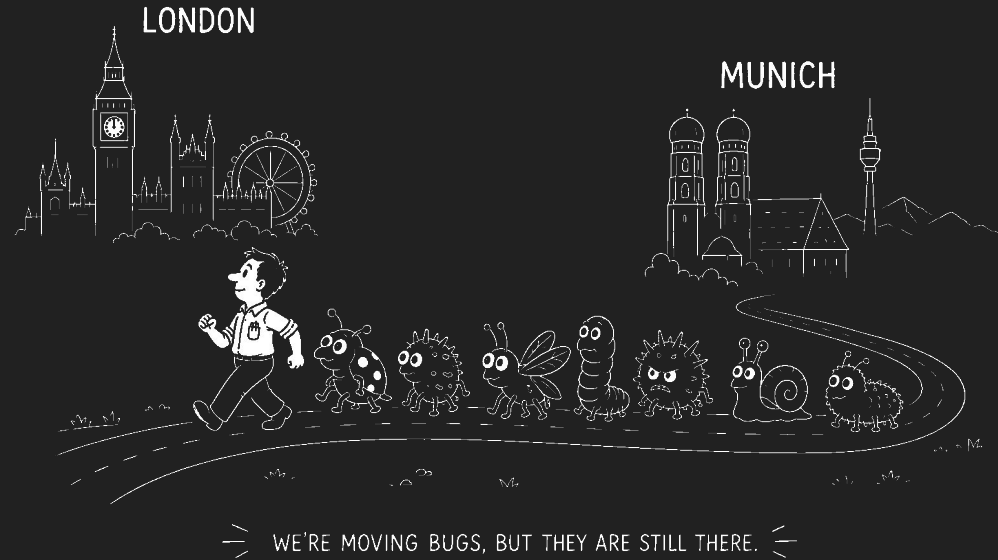
June 9-11, 2026

Co-hosted with Canonical

Motivation and Continuity

From Munich 2025 to London 2026

- 2025: Architectural investigation
- 2026: Experimental framework



Presentation Roadmap

- Why this matters
- Historical evidence
- The framework
- Demonstrations
- Toward safety metrics



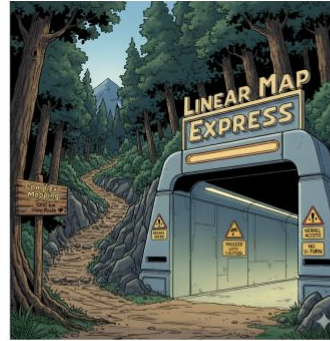
PART I: WHY THIS MATTERS



Safe VMA Concept Recap

- Isolation
- Determinism
- Ownership integrity
- Verification

The Linear Mapping Threat



- Introduction & Scope
- VMA Architecture: Lifecycle & Features
- **The Linear Mapping Threat**
- Safety Features: Defense & Detection
- Defining the Path to Functional Safety
- Epilogue

A Straight Tunnel Through Complexity... but at What Safety Cost?



ELISA
Enabling Linux in
Safety Applications

Aerospace · Automotive · Linux Features · Medical Devices · OS Engineering Process · Safety Architecture · Space Grade Linux · Systems · Tools

From ELISA Workshop 2025

Memory Ownership Invariants

- Read-only remains read-only
- Ownership metadata is accurate
- Reused memory is initialized
- Boundaries are preserved

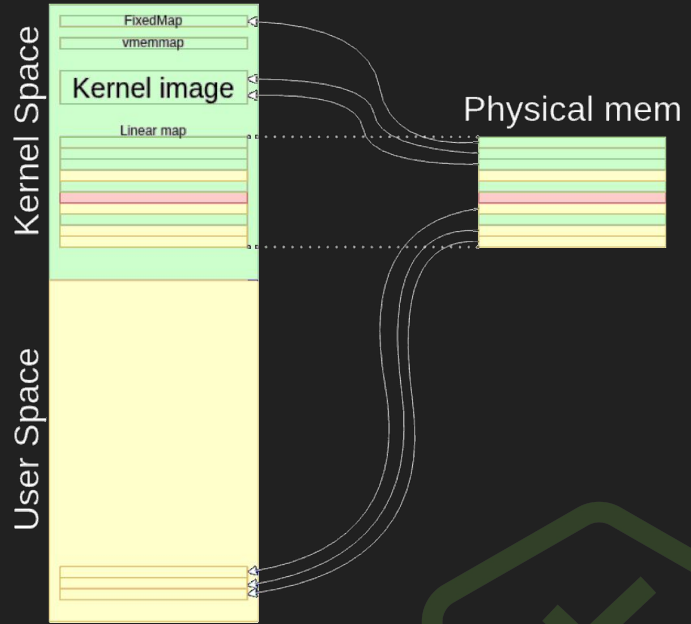


Physical proximity problem Recap

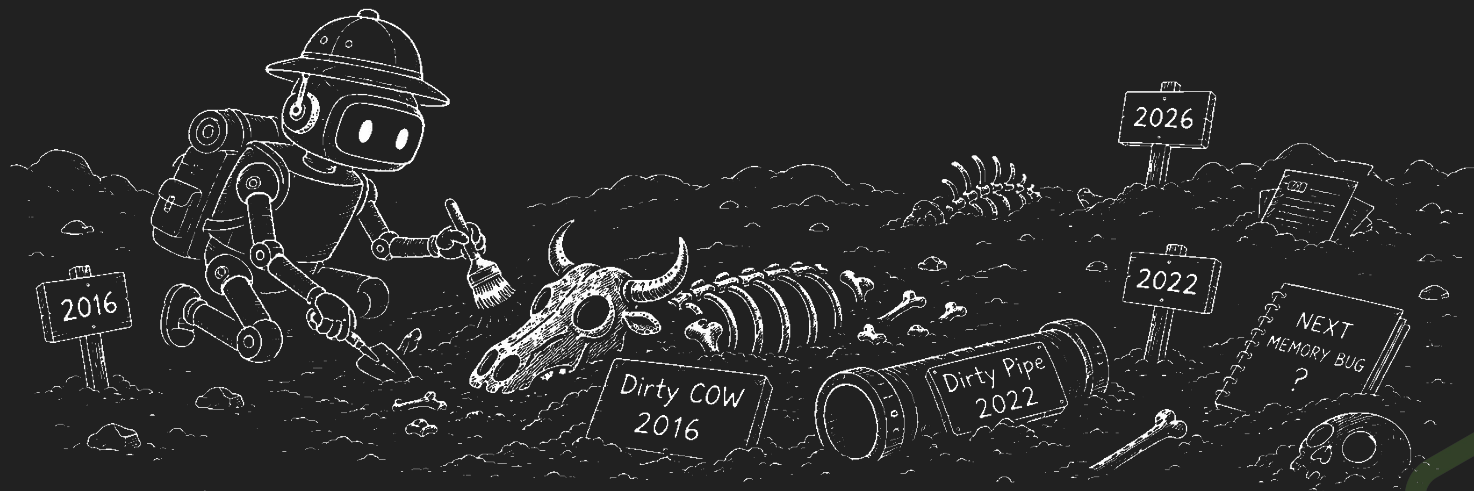
We have already seen that:

- The linear mapping covers all physical memory in a single kernel virtual address range.
- Kernel objects and user pages can end up physically adjacent.
- If that happens, a kernel bug may cause cross-domain corruption.

Is this just hand waving?



PART II: HISTORICAL EVIDENCE

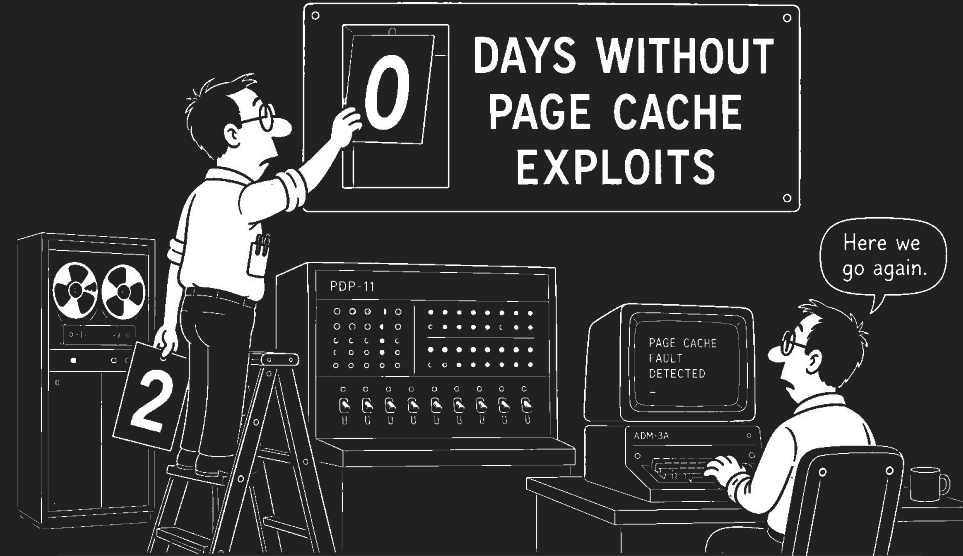


Old bugs do not disappear. We just get better at finding their fossils.



Timeline of Failures

- 2016 - Dirty COW
- 2022 - Dirty Pipe
- 2026 - Copy Fail
- 2026 - Dirty Frag
- 2026 - Fragnesia



—— Functional safety begins with assuming the counter will eventually return to zero. ——

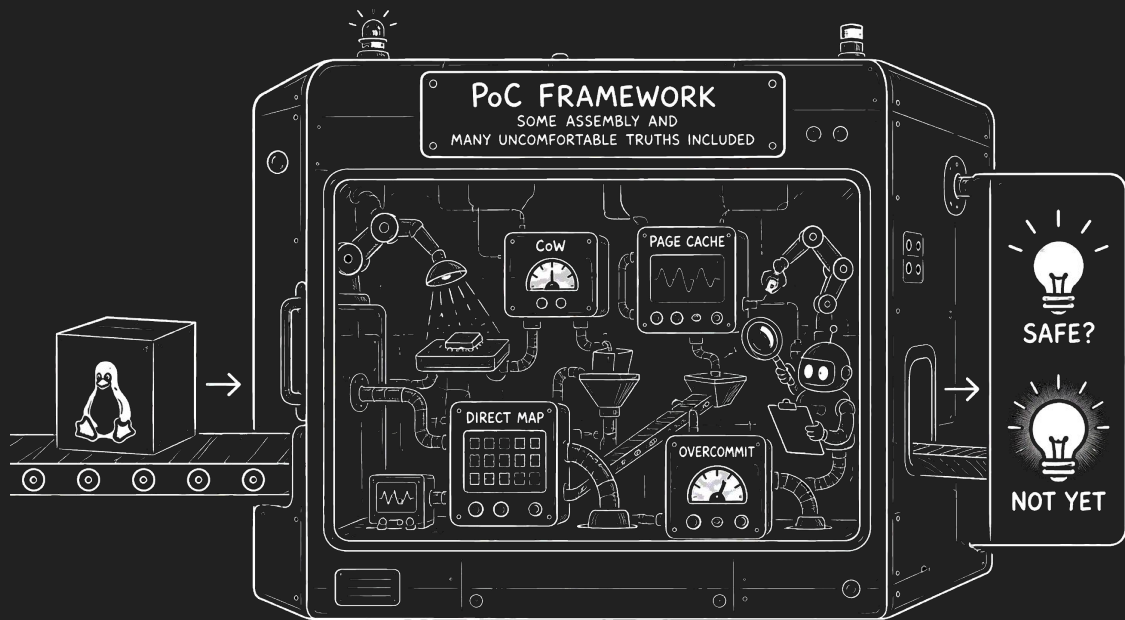


Common Broken Invariants

- Race
- Metadata reuse
- Copy semantics
- Fragment lifecycle



PART III: THE FRAMEWORK



Why a PoC Framework?

Turn isolated PoCs into a systematic methodology.

- Move from discussion to evidence.
- Reproducible experiments.
- Comparable results.



Framework Packaging

- Buildroot fork ([available](#))
- Yocto meta-layer (planned)

Ready-to-run environments for reproducible experiments

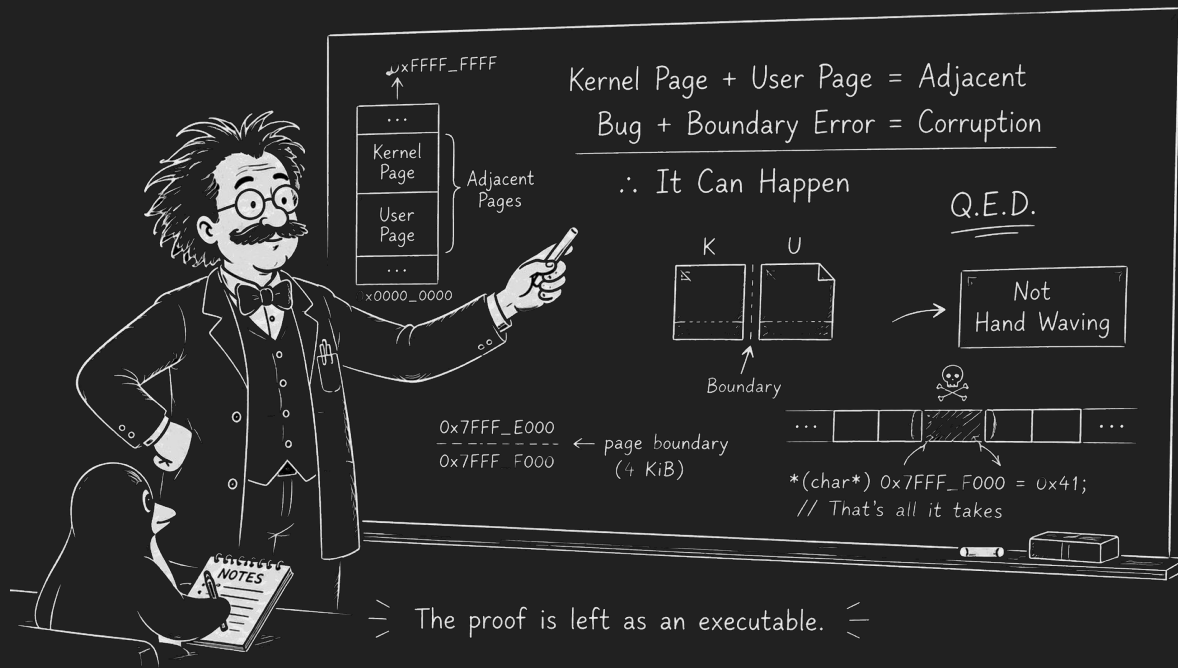


Framework Architecture

- Kernel patches
- Userspace tools
- Automation scripts
- Visualization
- Reports



PART IV: DEMONSTRATIONS



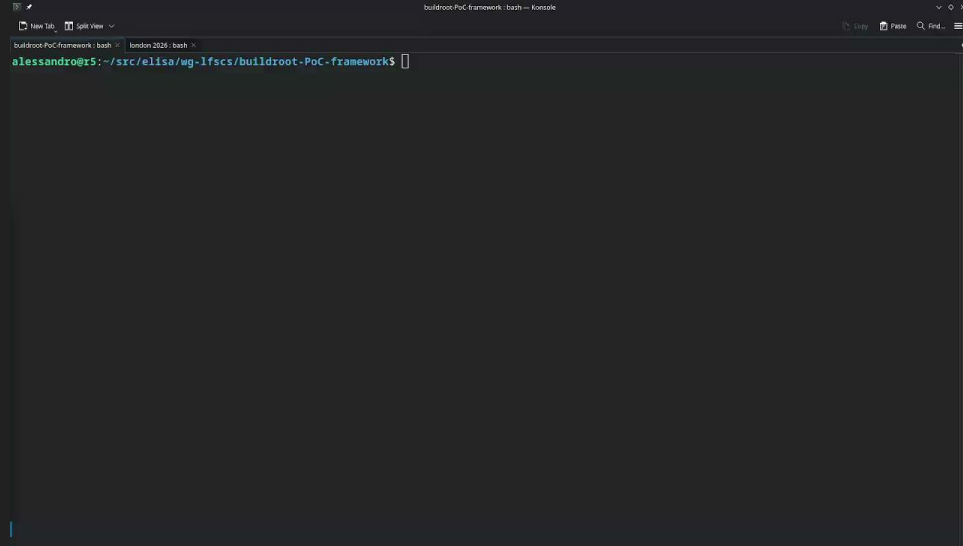
Existing PoCs Overview

All memory accesses are made equal... but some are more equal than others.	<code>vm_latency_demo</code>
My memory request succeeded, so everything is fine... right?	<code>overcommit_demo</code>
Kernel objects can sit right before a user page. Are we sure this can really happen?	<code>direct_map_view</code>
It can, it is really unfortunate. Now, what is the worst that could happen?	<code>umem_poke</code>



CoW and Lazy Allocation Latency

- A process starts and requests all the memory it needs.
- So everything should be ready, right?
- Not quite. The first access may trigger additional work before the memory is actually available.
- Linux applications are perfectly fine with this.

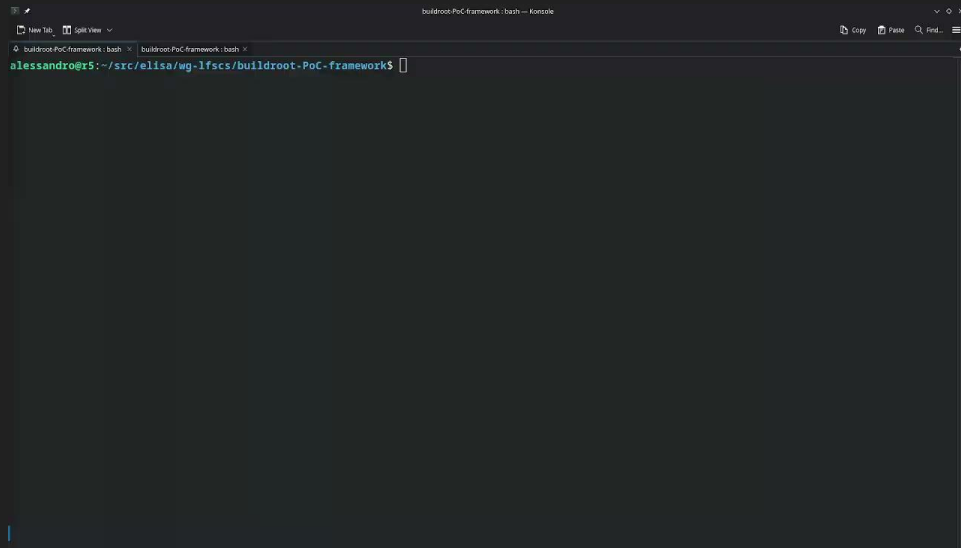


Is our safety-critical application fine with this too?



Overcommit Demonstration

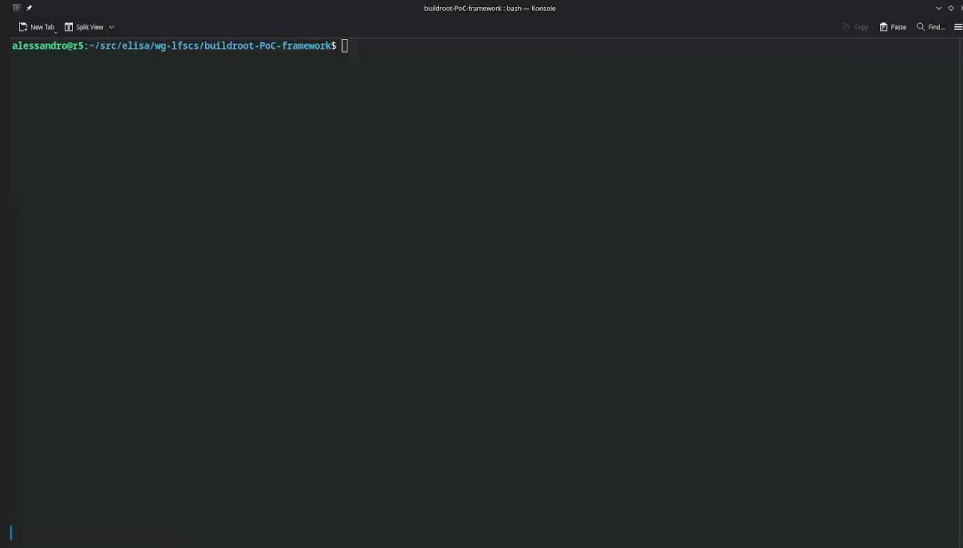
- A group of processes requests more memory than the system actually has.
- With `overcommit_memory=0`, everything starts normally. But failure happens when the processes begin using memory that does not really exist.
- With `overcommit_memory=2`, the same workload may not start at all.
- Changing the overcommit policy **does not solve** the problem entirely.
- Linux ecosystem assumes overcommit is available.

A screenshot of a terminal window titled 'buildroot-PoC-framework: bash - Konsole'. The terminal shows a shell prompt 'alessandro@5:~/src/elisa/wg-lfscs/buildroot-PoC-framework\$' with a cursor. The window has a dark theme and standard terminal controls at the top.

`fork` is a classic example of this.

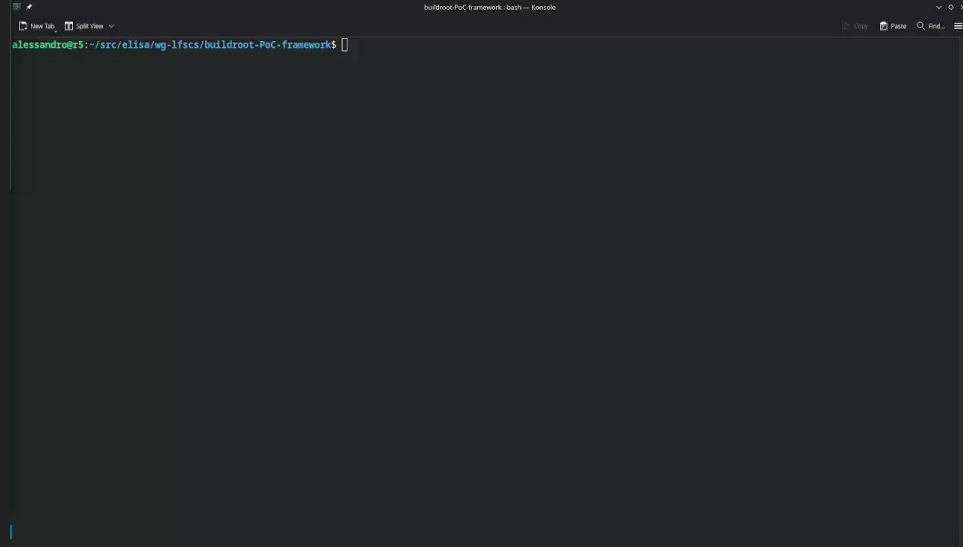
Direct Map Viewer - 1

- Shows the kernel's linear address map.
- Each character represents one physical page.
- If a **red** page appears immediately before an **orange** page, we have the condition we are looking for.
- To observe this, `direct_map_view` must first allocate memory and **touch** it.
- That is why the tool reports its memory usage as it runs.



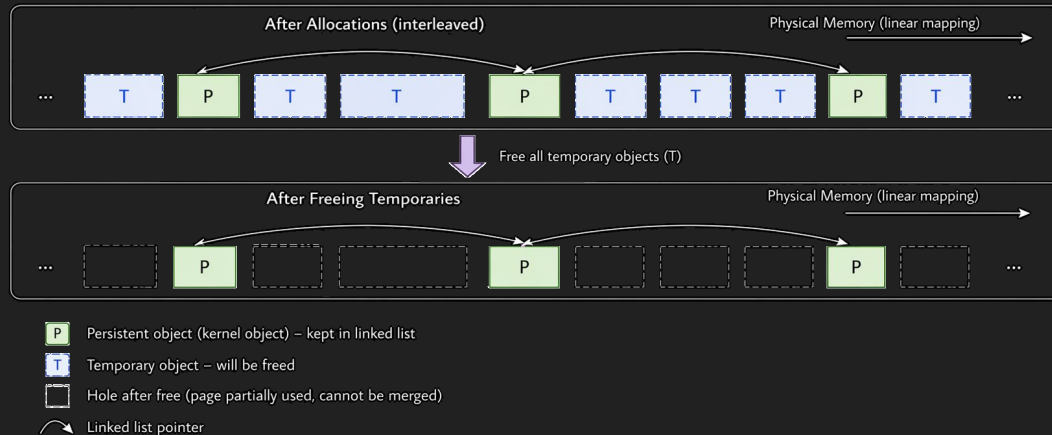
Direct Map Viewer - 2

- What we saw on x86 is significant.
- But within ELISA, we chose **ARM64** as our primary reference architecture.
- And memory management can vary significantly across architectures.



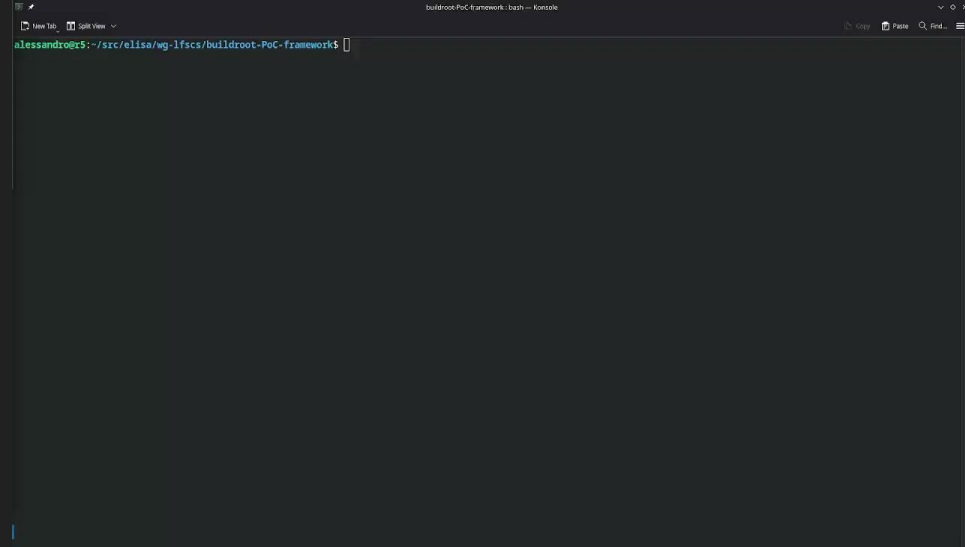
Is ARM64 Different?

- No... the system is simply idle.
- Idle system has no memory fragmentation.
- We can create fragmentation artificially using `mem_frag_ent`



Direct Map Viewer - 3

- Memory fragmentation changes the result.
- After introducing fragmentation, the condition appears on ARM64 as well.
- Kernel pages and user pages can become physically adjacent.
- But where exactly does the kernel data reside within that page?



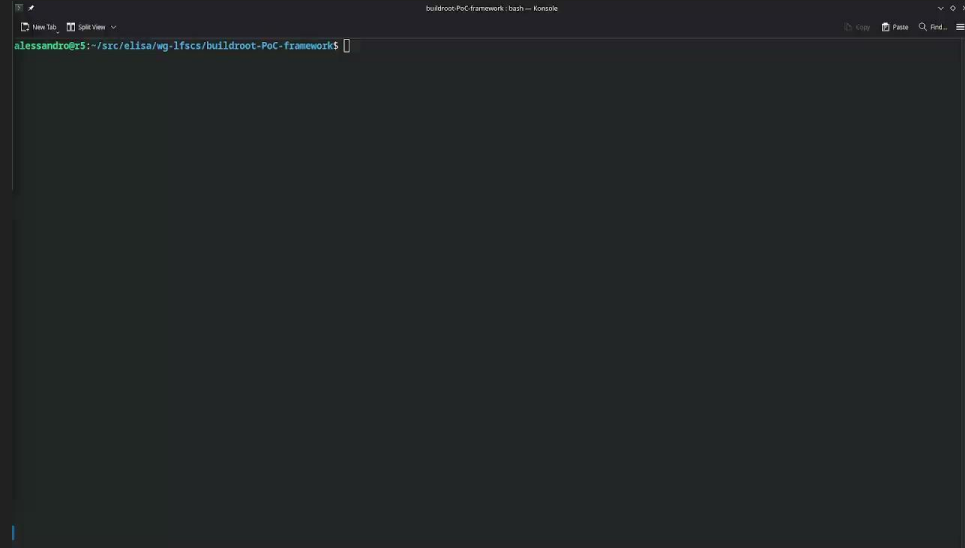
Direct Map Viewer - 4

Kernel data can reside very close to user-space memory.

- A small mistake in pointer arithmetic, and you are there.
- A small overflow, and you are there.

User-space memory is not as isolated as we might assume.

Now what?

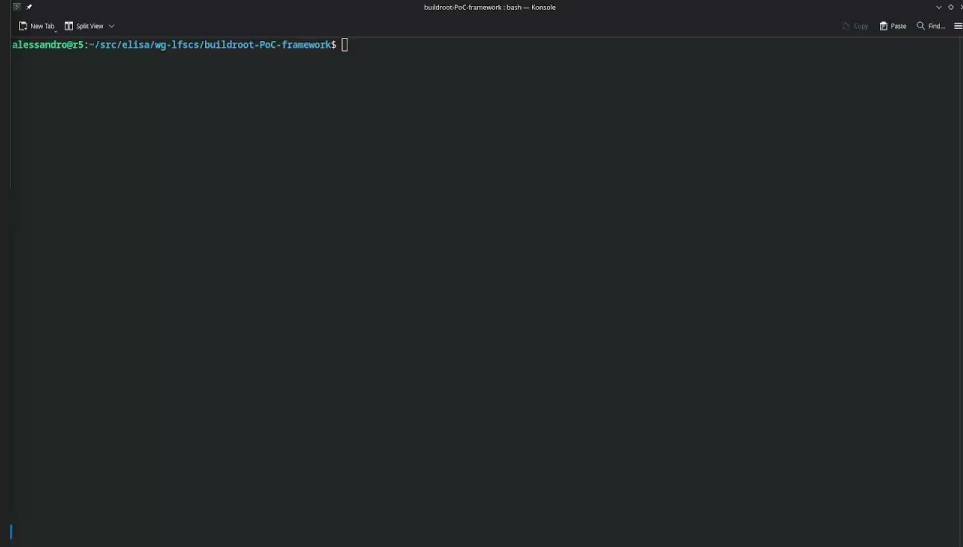


What is the worst that can happen?

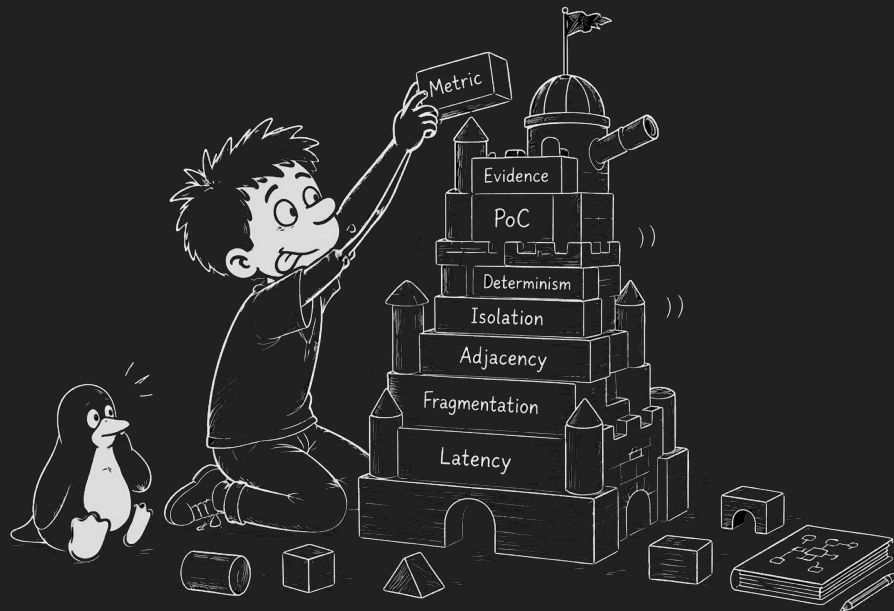
Cross-Domain Interference (umem_poke)

- I am not saying this will happen.
- I am not saying it is likely.
- I am saying that the worst-case scenario is exactly what functional safety seeks to avoid.

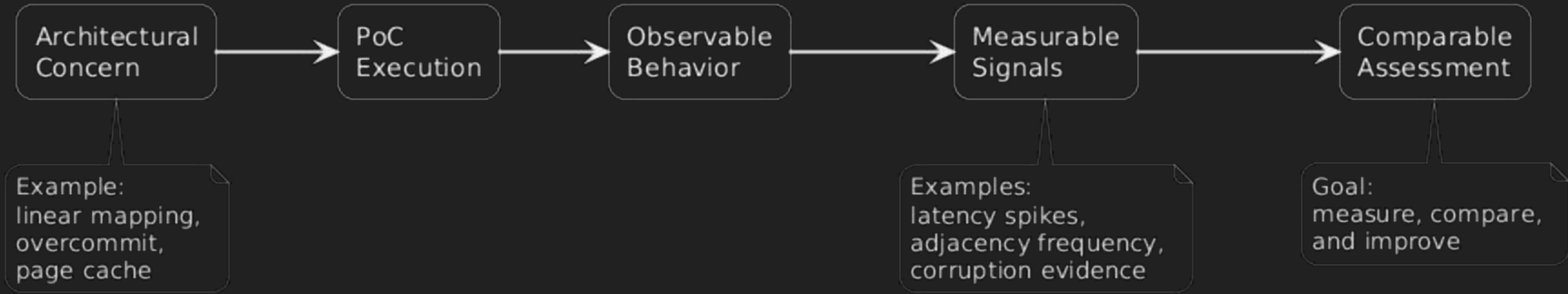
Target application kept running...
just differently.



PART V: FROM POC TO METRICS



From Hypothesis to Metrics



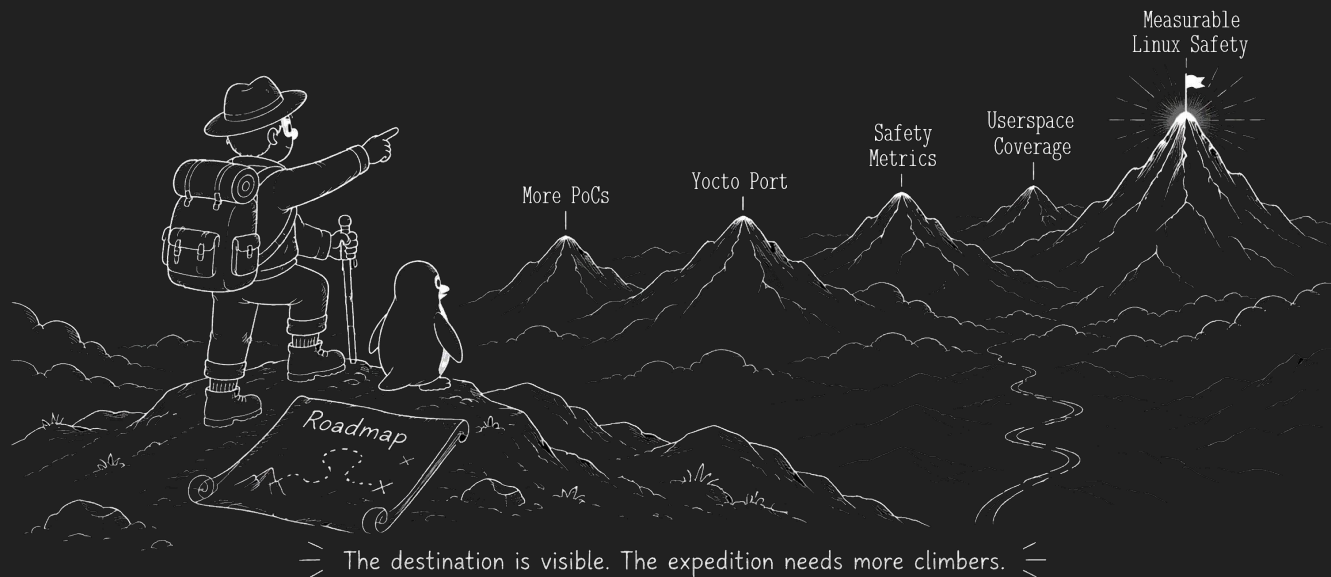
Candidate Metric Dimensions

- Isolation strength
- Timing determinism
- Memory ownership integrity
- Criticality observability
- Fragmentation sensitivity

Early ideas, not final metrics.



PART VI: ROADMAP



Toward Measurable Linux Safety

The question is no longer whether Linux memory isolation can fail...

The real question is whether we can measure and quantify those criticalities.

- Extend PoCs
- Support more architectures
- Add Yocto integration
- Define meaningful metrics
- Collaborate within ELISA



Toward Measurable Linux Safety

The question is no longer whether Linux memory isolation can fail...

The real question is whether we can measure and quantify those criticalities.

- Extend PoCs
- Support more architectures
- Add Yocto integration
- Define meaningful metrics
- Collaborate within ELISA

ADVERTISEMENT



Thank You / Q&A

